# Lego Power Functions Wireless Control System

Build Guide

| Component | Description |
|---|---|
| Train Receiver | 1-2 motor controller, 2S LiPo, USB-C charging |
| Handheld Controller | Knob speed, button direction, AA batteries |
| Communication | ESP-NOW (WiFi-based, low latency) |
| Pairing | Button-based, configs saved to NVS flash |
| Enclosure | Train: 8×4 stud Lego box, Controller: project box |

Version 1.1 — December 2024

# Part 1: Train Receiver

## Parts List

| Part | Description | Qty | Price |
|---|---|---|---|
| ESP32-C3 SuperMini | Microcontroller with WiFi/BLE | 1 | $2-4 |
| TB6612FNG Module | Dual H-bridge motor driver | 1 | $3-5 |
| USB-C 2S BMS (balanced) | Charger + balancer + protection | 1 | $2-3 |
| 2S LiPo 350mAh | 45×15×14mm, with balance lead | 1 | $8-12 |
| AMS1117-3.3 Module | 3.3V voltage regulator | 1 | $0.50-1 |
| Mini Slide Switch | SPST power switch, 3A+ rated | 1 | $0.50 |
| Momentary Pushbutton | Pairing button (or use onboard BOOT) | 1 | $0.20-0.50 |
| Lego PF Extension Cable | Cut & splice for connectors | 2 | $2-4 ea |
| Prototype board / wires | For assembly | 1 | $2-3 |

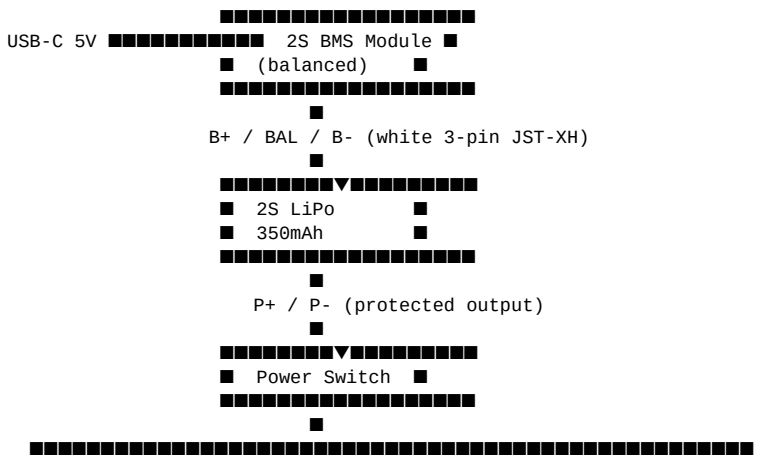
**Estimated Total: $26-36**

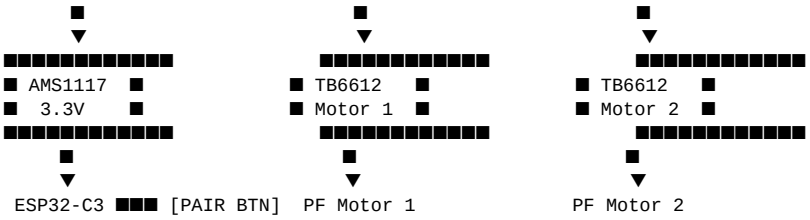
**Note:** Many ESP32-C3 SuperMini boards have a BOOT button on GPIO9 — you can use that for pairing instead of adding an external button. Check your board first.

### Suggested Sources

- **ESP32-C3 SuperMini:** AliExpress, Amazon
- **TB6612FNG Module:** SparkFun, Pololu, AliExpress
- **USB-C 2S BMS:** AliExpress (search "Type-C USB 2S BMS balanced")
- **2S LiPo:** Amazon, AliExpress (must have 3-pin balance connector)
- **PF Extension Cables:** BrickLink, AliExpress, or official Lego

## System Architecture




AMS1117
3.3V
TB6612
Motor 1
TB6612
Motor 2
ESP32-C3
[PAIR BTN]
PF Motor 1
PF Motor 2

# GPIO Assignments

| Function | GPIO | Notes |
|---|---|---|
| Motor 1 PWM | GPIO2 | Speed control |
| Motor 1 Direction | GPIO3 | TB6612 AIN1 |
| Motor 2 PWM | GPIO4 | Speed control |
| Motor 2 Direction | GPIO5 | TB6612 BIN1 |
| Pairing Button | GPIO9 | BOOT button on most boards |

# Wiring Connections

## Power Wiring

| From | To | Notes |
|---|---|---|
| Battery 3-pin JST-XH | BMS: B+, BAL, B- | White connector |
| BMS P+ | Switch input | Red wire |
| BMS P- | Common GND rail | Black wire |
| Switch output | TB6612 VM | Switched battery power |
| Switch output | AMS1117 VIN | Switched battery power |
| AMS1117 VOUT | ESP32 3V3 | 3.3V regulated |
| AMS1117 VOUT | TB6612 VCC | 3.3V logic power |
| AMS1117 VOUT | TB6612 STBY | Enable driver |
| GND | All GND pins | Common ground |

## Control Signal Wiring

| From | To | Notes |
|---|---|---|
| ESP32 GPIO2 | TB6612 PWMA | Motor 1 speed (PWM) |
| ESP32 GPIO3 | TB6612 AIN1 | Motor 1 direction |
| GND | TB6612 AIN2 | Tie to ground |
| ESP32 GPIO4 | TB6612 PWMB | Motor 2 speed (PWM) |
| ESP32 GPIO5 | TB6612 BIN1 | Motor 2 direction |
| GND | TB6612 BIN2 | Tie to ground |
| Pair Button | ESP32 GPIO9 | Or use onboard BOOT button |
| Pair Button | GND | Other side of button |

## Motor Output Wiring

| From | To | Notes |
|---|---|---|

| | | |
|---|---|---|
| TB6612 AO1 | PF Cable 1 - C1 | Motor 1 + |
| TB6612 AO2 | PF Cable 1 - C2 | Motor 1 - |
| TB6612 BO1 | PF Cable 2 - C1 | Motor 2 + |
| TB6612 BO2 | PF Cable 2 - C2 | Motor 2 - |

# Lego PF Cable Pinout

When cutting a PF extension cable, keep the end that plugs into the motor. Use a multimeter to identify the wires — colors vary by manufacturer.

| Pin | Signal | Connect To |
|---|---|---|
| 1 | 9V | Not used |
| 2 | C1 | TB6612 xO1 |
| 3 | C2 | TB6612 xO2 |
| 4 | GND | Common GND |

# Assembly Steps

**1. Prepare the PF Cables**

Cut the PF extension cable, keeping the motor end. Strip wires and identify C1, C2, GND with a multimeter.

**2. Wire the Power Section**

Connect battery balance lead to BMS. Connect BMS P+ to switch, switch output to TB6612 VM and AMS1117 VIN.

**3. Wire the ESP32**

Connect 3.3V rail to ESP32 3V3, GND to ESP32 GND. Verify power before proceeding.

**4. Wire the TB6612**

VM to battery, VCC and STBY to 3.3V, AIN2/BIN2 to GND. Connect control signals from ESP32.

**5. Add Pairing Button**

Connect button between GPIO9 and GND (or use onboard BOOT button if available).

**6. Wire the Motors**

Connect AO1/AO2 to Motor 1 C1/C2. Connect BO1/BO2 to Motor 2 C1/C2.

**7. Test**

Verify voltages, upload firmware, test pairing and motor control.

# Part 2: Handheld Controller

A simple handheld controller with a knob for speed and a button to toggle direction. Powered by 2×AA batteries for simplicity — no charging required. The same button is used for direction toggle (short press) and pairing mode (hold on boot).

## Parts List

| Part | Description | Qty | Price |
|---|---|---|---|
| ESP32-C3 SuperMini | Microcontroller with WiFi/BLE | 1 | $2-4 |
| 10K Potentiometer | Linear taper, panel mount | 1 | $1-2 |
| Knob | For potentiometer shaft | 1 | $0.50-1 |
| Momentary Pushbutton | Direction toggle + pairing | 1 | $0.50-1 |
| 2×AA Battery Holder | 3V power source | 1 | $1-2 |
| Slide Switch (SPST) | Power on/off | 1 | $0.50 |
| LED + 1K Resistor | Direction indicator | 1 | $0.20 |
| Project Box | Small enclosure | 1 | $2-5 |

**Estimated Total: $8-15**

## Schematic

## Wiring Connections

| From | To | Notes |
|---|---|---|
| Battery + | Switch input | Red wire |
| Switch output | ESP32 3V3 | Switched power |
| Battery - | ESP32 GND | Common ground |
| Pot pin 1 | GND | Ground reference |
| Pot pin 2 (wiper) | ESP32 GPIO0 | Analog input |
| Pot pin 3 | 3V3 | Voltage reference |
| Button pin 1 | ESP32 GPIO1 | Direction + pairing |
| Button pin 2 | GND | Uses internal pull-up |
| LED anode | ESP32 GPIO2 | Through 1K resistor |
| LED cathode | GND | |

## Assembly Steps

**1. Prepare the Enclosure**

Drill holes for potentiometer, button, power switch, and LED.

**2. Mount Components**

Install pot, button, switch, and LED in the enclosure.

**3. Wire Power**

Connect battery holder through switch to ESP32 3V3 and GND.

**4. Wire Controls**

Connect pot wiper to GPIO0, button to GPIO1, LED to GPIO2.

**5. Install Battery Holder**

Secure inside enclosure with double-sided tape or screws.

**6. Test**

Verify pot reads 0-4095, button toggles LED, pairing mode works.

## Operation

- **Speed:** Turn knob — full CCW = stop, full CW = max speed
- **Direction:** Short press button to toggle forward/reverse
- **LED:** On = forward, Off = reverse
- **Pairing:** Hold button while turning on power (see Part 4)

# Part 3: Pairing Mode

Both devices store their paired partner's MAC address in non-volatile storage (NVS). Once paired, they remember each other across power cycles — no need to pair again unless you want to switch to a different train or controller.

## Pairing Procedure

**1. Put Train in Pairing Mode**

Hold the pairing button (GPIO9 / BOOT) while turning on the train. Motors will twitch briefly to confirm pairing mode.

**2. Put Controller in Pairing Mode**

Hold the direction button while turning on the controller. LED will blink rapidly to confirm pairing mode.

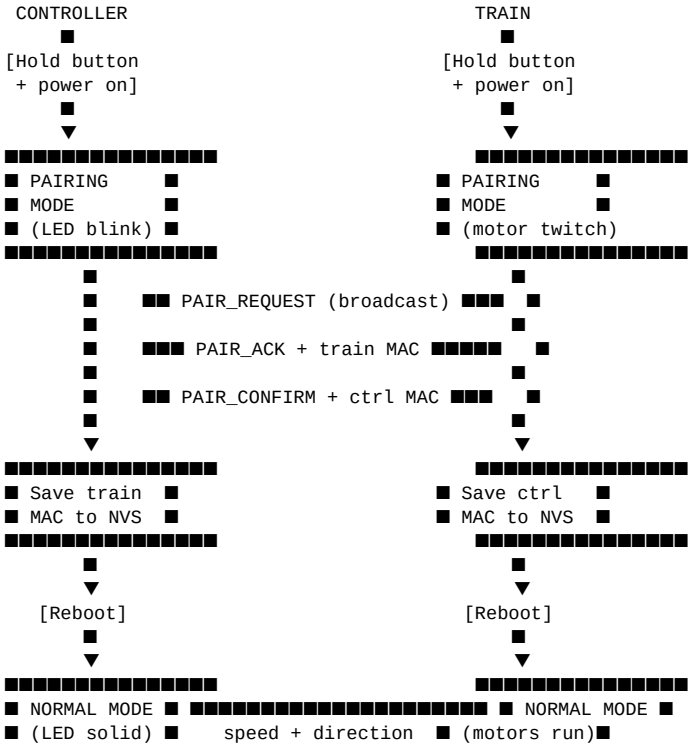
**3. Wait for Pairing**

Devices will find each other automatically (1-3 seconds). Train motors twitch twice, controller LED goes solid when paired.

**4. Done!**

Both devices save the pairing and reboot into normal mode. They will reconnect automatically on future power-ups.

### Pairing Flow Diagram



## Stored Configuration

The following settings are saved to NVS flash and persist across power cycles:

| Device | Setting | Description |
|---|---|---|
| Train | Controller MAC | Only accepts commands from this controller |
| Train | Motor 1 Invert | Flip motor 1 direction (future feature) |
| Train | Motor 2 Invert | Flip motor 2 direction (future feature) |
| Train | Max Speed | Speed limit 0-255 (future feature) |
| Controller | Train MAC | Sends commands to this train only |

# Part 4: Firmware

Both units use ESP-NOW for communication — a fast, connectionless protocol. Pairing info and config are stored in NVS flash.

## Controller Firmware

```
#include <esp_now.h>
#include <WiFi.h>
#include <Preferences.h>

#define POT_PIN    0
#define BTN_PIN    1
#define LED_PIN    2

Preferences prefs;
uint8_t trainMAC[6] = {0xFF,0xFF,0xFF,0xFF,0xFF,0xFF};
bool paired = false;
bool forward = true;
bool pairingMode = false;

enum MsgType { MSG_CONTROL, MSG_PAIR_REQ, MSG_PAIR_ACK, MSG_PAIR_CONFIRM };

typedef struct {
  uint8_t type;
  uint8_t speed;
  bool forward;
  uint8_t mac[6];
} Message;

Message msg;

void loadConfig() {
  prefs.begin("ctrl", false);
  if (prefs.getBool("paired", false)) {
    paired = true;
    trainMAC[0] = prefs.getUChar("mac0", 0xFF);
    trainMAC[1] = prefs.getUChar("mac1", 0xFF);
    trainMAC[2] = prefs.getUChar("mac2", 0xFF);
    trainMAC[3] = prefs.getUChar("mac3", 0xFF);
    trainMAC[4] = prefs.getUChar("mac4", 0xFF);
    trainMAC[5] = prefs.getUChar("mac5", 0xFF);
  }
}

void saveTrainMAC(uint8_t *mac) {
  prefs.putBool("paired", true);
  prefs.putUChar("mac0", mac[0]); prefs.putUChar("mac1", mac[1]);
  prefs.putUChar("mac2", mac[2]); prefs.putUChar("mac3", mac[3]);
  prefs.putUChar("mac4", mac[4]); prefs.putUChar("mac5", mac[5]);
}

void onReceive(const esp_now_recv_info_t *info, const uint8_t *data, int len) {
  if (len < 1) return;
  Message *m = (Message*)data;
  if (pairingMode && m->type == MSG_PAIR_ACK) {
    memcpy(trainMAC, m->mac, 6);
    saveTrainMAC(trainMAC);
    // Send confirm
    msg.type = MSG_PAIR_CONFIRM;
    WiFi.macAddress(msg.mac);
    esp_now_send(trainMAC, (uint8_t*)&msg, sizeof(msg));
    delay(100);
    ESP.restart();
  }
}

void setup() {
  pinMode(BTN_PIN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);

  // Check for pairing mode (button held at boot)
  if (digitalRead(BTN_PIN) == LOW) {
    pairingMode = true;
  }
```

```
  loadConfig();

  WiFi.mode(WIFI_STA);
  esp_now_init();
  esp_now_register_recv_cb(onReceive);

  // Add peer (broadcast for pairing, specific MAC for normal)
  esp_now_peer_info_t peer = {};
  memcpy(peer.peer_addr, trainMAC, 6);
  peer.channel = 0;
  peer.encrypt = false;
  esp_now_add_peer(&peer);

  if (pairingMode) {
    // Blink LED rapidly in pairing mode
    for(int i=0; i<10; i++) {
      digitalWrite(LED_PIN, i%2);
      delay(100);
    }
  }
  digitalWrite(LED_PIN, forward);
}

bool lastBtn = HIGH;
unsigned long lastSend = 0;

void loop() {
  // Pairing mode: send pair requests
  if (pairingMode) {
    digitalWrite(LED_PIN, (millis()/100) % 2);  // Fast blink
    msg.type = MSG_PAIR_REQ;
    WiFi.macAddress(msg.mac);
    uint8_t broadcast[6] = {0xFF,0xFF,0xFF,0xFF,0xFF,0xFF};
    esp_now_send(broadcast, (uint8_t*)&msg, sizeof(msg));
    delay(200);
    return;
  }

  // Normal mode
  int raw = analogRead(POT_PIN);
  msg.type = MSG_CONTROL;
  msg.speed = map(raw, 0, 4095, 0, 255);

  bool btn = digitalRead(BTN_PIN);
  if (btn == LOW && lastBtn == HIGH) {
    forward = !forward;
    digitalWrite(LED_PIN, forward);
  }
  lastBtn = btn;
  msg.forward = forward;

  if (millis() - lastSend > 50) {
    esp_now_send(trainMAC, (uint8_t*)&msg, sizeof(msg));
    lastSend = millis();
  }
}
```

# Train Receiver Firmware

```
#include <esp_now.h>
#include <WiFi.h>
#include <Preferences.h>

#define M1_PWM  2
#define M1_DIR  3
#define M2_PWM  4
#define M2_DIR  5
#define PAIR_BTN 9

Preferences prefs;
uint8_t ctrlMAC[6] = {0xFF,0xFF,0xFF,0xFF,0xFF,0xFF};
bool paired = false;
bool pairingMode = false;
unsigned long lastRecv = 0;

enum MsgType { MSG_CONTROL, MSG_PAIR_REQ, MSG_PAIR_ACK, MSG_PAIR_CONFIRM };

typedef struct {
  uint8_t type;
  uint8_t speed;
  bool forward;
  uint8_t mac[6];
} Message;

Message msg;

void loadConfig() {
  prefs.begin("train", false);
  if (prefs.getBool("paired", false)) {
    paired = true;
    ctrlMAC[0] = prefs.getUChar("mac0", 0xFF);
    ctrlMAC[1] = prefs.getUChar("mac1", 0xFF);
    ctrlMAC[2] = prefs.getUChar("mac2", 0xFF);
    ctrlMAC[3] = prefs.getUChar("mac3", 0xFF);
    ctrlMAC[4] = prefs.getUChar("mac4", 0xFF);
    ctrlMAC[5] = prefs.getUChar("mac5", 0xFF);
  }
}

void saveCtrlMAC(uint8_t *mac) {
  prefs.putBool("paired", true);
  prefs.putUChar("mac0", mac[0]); prefs.putUChar("mac1", mac[1]);
  prefs.putUChar("mac2", mac[2]); prefs.putUChar("mac3", mac[3]);
  prefs.putUChar("mac4", mac[4]); prefs.putUChar("mac5", mac[5]);
}

void setMotor(int pwm, int dir, uint8_t speed, bool fwd) {
  digitalWrite(dir, fwd ? HIGH : LOW);
  analogWrite(pwm, speed);
}

void motorTwitch(int count) {
  for(int i=0; i<count; i++) {
    setMotor(M1_PWM, M1_DIR, 100, true);
    delay(100);
    setMotor(M1_PWM, M1_DIR, 0, true);
    delay(100);
  }
}

void onReceive(const esp_now_recv_info_t *info, const uint8_t *data, int len) {
  if (len < 1) return;
  Message *m = (Message*)data;

  if (pairingMode && m->type == MSG_PAIR_REQ) {
    // Send ACK with our MAC
    msg.type = MSG_PAIR_ACK;
    WiFi.macAddress(msg.mac);
    esp_now_send(info->src_addr, (uint8_t*)&msg, sizeof(msg));
  }
  else if (pairingMode && m->type == MSG_PAIR_CONFIRM) {
    memcpy(ctrlMAC, m->mac, 6);
    saveCtrlMAC(ctrlMAC);
    motorTwitch(2);  // Confirm pairing
    delay(500);
    ESP.restart();
  }
  else if (!pairingMode && m->type == MSG_CONTROL) {
    // In normal mode, only accept from paired controller
```

```
    if (paired && memcmp(info->src_addr, ctrlMAC, 6) != 0) return;
    setMotor(M1_PWM, M1_DIR, m->speed, m->forward);
    setMotor(M2_PWM, M2_DIR, m->speed, m->forward);
    lastRecv = millis();
  }
}

void setup() {
  pinMode(M1_PWM, OUTPUT); pinMode(M1_DIR, OUTPUT);
  pinMode(M2_PWM, OUTPUT); pinMode(M2_DIR, OUTPUT);
  pinMode(PAIR_BTN, INPUT_PULLUP);

  if (digitalRead(PAIR_BTN) == LOW) {
    pairingMode = true;
  }

  loadConfig();

  WiFi.mode(WIFI_STA);
  esp_now_init();
  esp_now_register_recv_cb(onReceive);

  if (pairingMode) {
    motorTwitch(1);  // Signal pairing mode
  }
}

void loop() {
  // Safety: stop if no signal for 500ms
  if (!pairingMode && millis() - lastRecv > 500) {
    setMotor(M1_PWM, M1_DIR, 0, true);
    setMotor(M2_PWM, M2_DIR, 0, true);
  }
  delay(20);
}
```

# Firmware Upload Instructions

1. **Install Arduino IDE** and add ESP32 board support:
   File → Preferences → Additional Board URLs:
   `https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json`

2. **Select board:** Tools → Board → ESP32C3 Dev Module

3. **Upload train firmware first** — note: upload order doesn't matter for pairing

4. **Upload controller firmware**

5. **Follow pairing procedure** (Part 3) to link the devices

## Troubleshooting

| Problem | Possible Cause | Solution |
|---|---|---|
| No power (train) | Switch off, dead battery | Check switch, charge via USB-C |
| No power (controller) | Dead batteries, switch off | Replace AA batteries |
| Pairing fails | Not in pairing mode | Hold button WHILE powering on |
| Pairing fails | Out of range | Move devices closer together |
| Motor not spinning | Wrong wiring, STBY low | Verify connections, STBY to 3.3V |
| Wrong direction | C1/C2 swapped | Swap AO1/AO2 wires |
| Train stops randomly | Signal lost > 500ms | Check controller batteries, range |
| Unpaired after reboot | NVS not saving | Re-pair, check for code errors |

## Safety Notes

• **LiPo batteries can be dangerous** — never puncture, short, or overcharge

• **Always use the BMS** — provides overcharge, over-discharge, and short-circuit protection

• **Don't leave charging unattended** — especially during initial tests

• **Disconnect battery** when making wiring changes

• **Check polarity** before connecting anything

• **Supervise children** when using the controller

## Runtime Estimates

**Train Receiver (350mAh 2S LiPo):**

| Load | Current | Runtime |
|---|---|---|
| 1 motor, light | ~100mA | ~3+ hours |
| 2 motors, moderate | ~300mA | ~1 hour |
| 2 motors, heavy | ~500mA | ~40 min |

**Controller (2×AA):** 20-40+ hours typical (low power between transmits)

## Expansion Options

• **4 motors:** Add second TB6612FNG, use GPIO6/7/8/10
• **Multiple trains:** Add train select button to controller, store multiple MACs
• **Lights/sound:** Use spare GPIOs on train receiver
• **Phone app:** Replace controller with BLE smartphone app
• **Speed limit:** Add max speed config in NVS for younger kids